

6. 外部主控二次开发

6. 外部主控二次开发

6.1 树莓派网络控制机械臂

6.1.1 项目介绍

6.1.2 实现流程

6.1.3 程序下载

6.1.4 实现效果

6.1.5 程序分析

6.2 树莓派串口控制机械臂

6.2.1 项目介绍

6.2.2 实现流程

6.2.3 程序下载

6.2.4 实现效果

6.2.5 程序分析

6.3 Jetson网络控制机械臂

6.3.1 项目介绍

6.3.2 实现流程

6.3.3 程序下载

6.3.4 实现效果

6.3.5 程序分析

6.4 Jetson串口控制机械臂

6.4.1 项目介绍

6.4.2 实现流程

6.4.3 程序下载

6.4.4 实现效果

6.4.5 程序分析

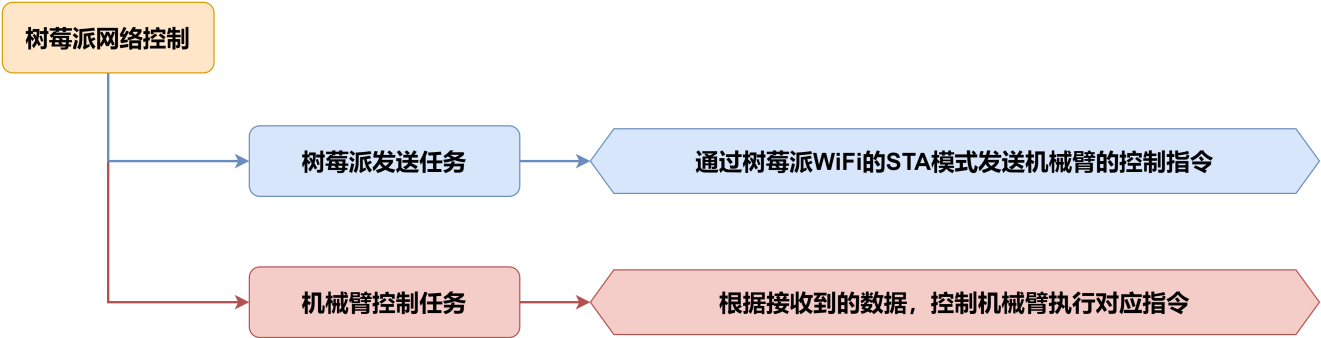
外部主控的网络、串口控制都遵循同一套控制协议，本章以坐标控制指令为例，若想参考更多控制指令，可前往“**3.运动控制**”章节内的“**3.3 串口协议解析**”中查看。其中对于不熟悉树莓派和Jetson的用户，可以参考本文档同目录下的“**03 远程连接教程**”文件夹。

6.1 树莓派网络控制机械臂

6.1.1 项目介绍

本节通过树莓派的 Python 程序连接 NexArm 热点，并通过 TCP 网络向机械臂发送控制指令。程序运行后，树莓派先检测当前无线网络状态，在确认已经连接到 NexArm 热点后，再建立与机械臂控制端口的通信连接，随后按照预设坐标序列依次下发运动指令，实现机械臂的定点夹取与搬运演示。

6.1.2 实现流程



6.1.3 程序下载

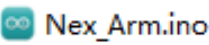
• NexArm程序下载

机械臂在进行外设主控控制功能前，需要先下载对应的控制程序，程序下载可参考以下步骤：

- 1. 使用Type-C数据线将NexArm与电脑进行连接。



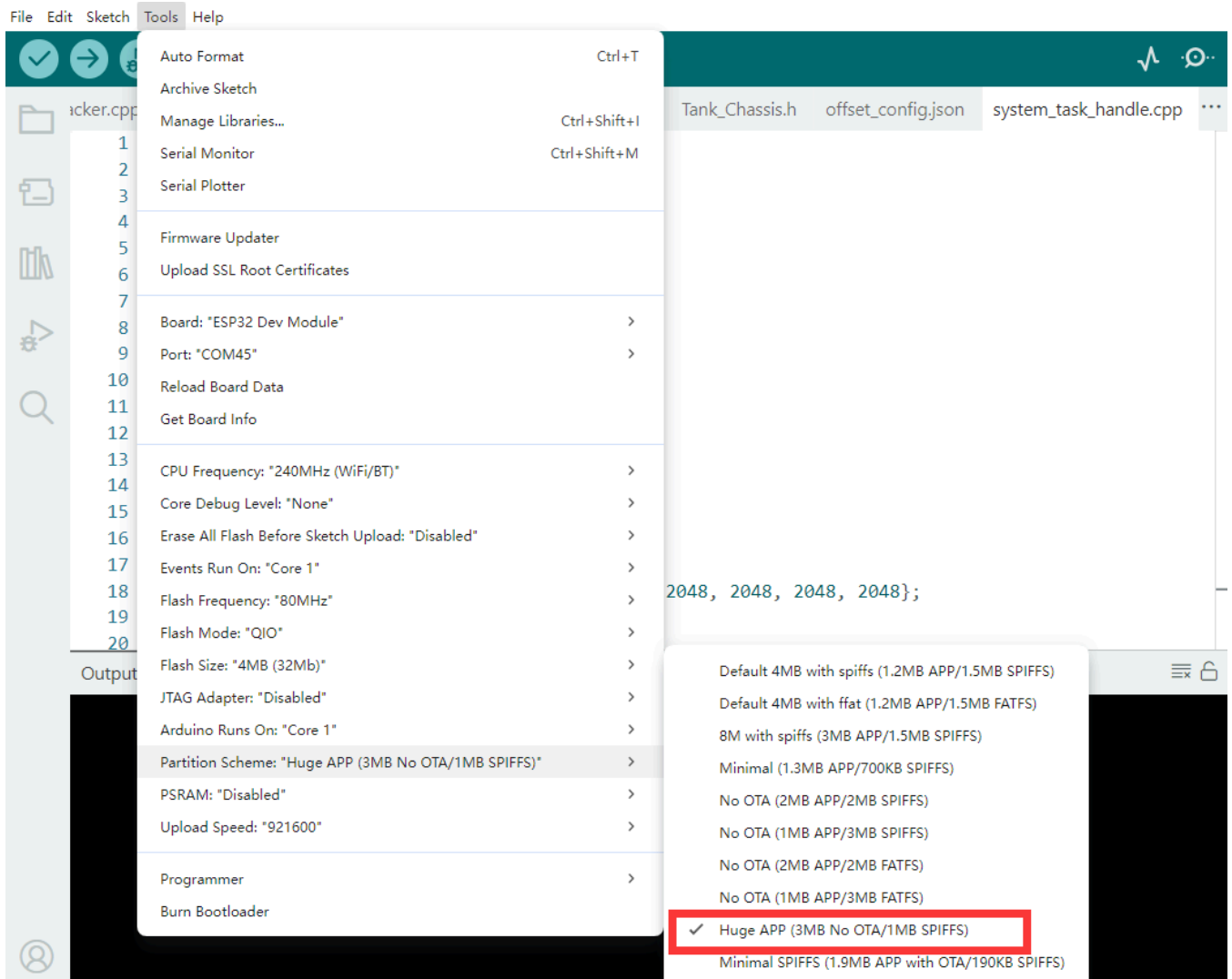
- 2. 打开本文档同路径下"源码"中对应的ino程序文件。



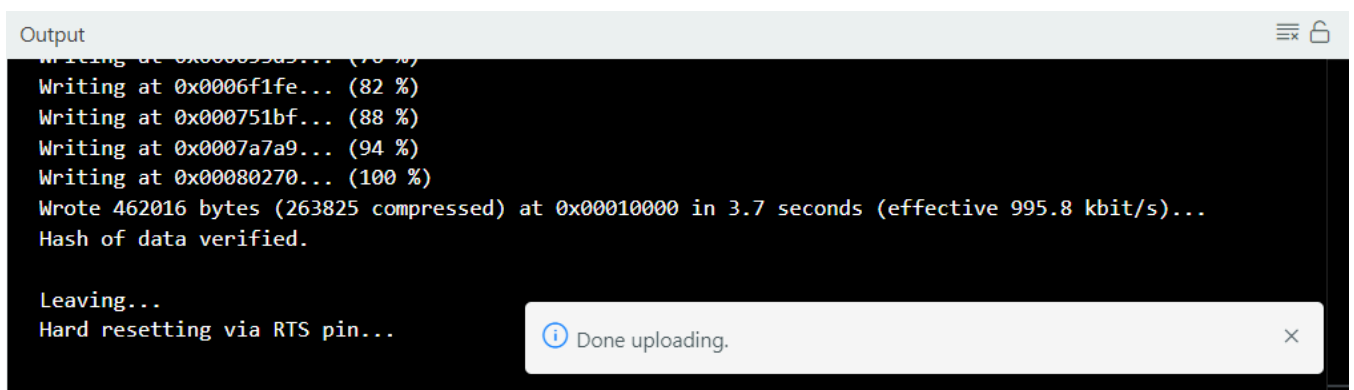
- 3. 打开后，选择开发板型号，具体型号如下图所示：



- 4. 点击菜单栏的"工具"，按照下图选择相应的ESP32的开发板配置：



5. 先点击"编译", 再点击"上传", 当软件下方输出框出现如下界面, 说明程序下载成功:



• 树莓派程序下载

使用远程工具将"源码\WiFi_Control"文件夹, 上传至树莓派中。

6.1.4 实现效果

1. 首先使用网线连接树莓派, 并在终端输入 `sudo ifconfig` 指令查找 `eth0` 的IP地址, 并使用远程工具通过有线IP地址进行连接。

```
> sudo ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.11.138 netmask 255.255.254.0 broadcast 192.168.11.255
    inet6 fe80::84ee:1627:16f2:b04c prefixlen 64 scopeid 0x20<link>
    ether 2c:cf:67:68:a9:37 txqueuelen 1000 (Ethernet)
    RX packets 10812 bytes 1508077 (1.4 MiB)
    RX errors 0 dropped 153 overruns 0 frame 0
    TX packets 741 bytes 94625 (92.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 106
```

2. 接着输入 `nmcli device status` 指令查询 `wlan0` 的连接状态，其中 "WN-25011421" 为网络连接名称。

```
> nmcli device status
DEVICE      TYPE      STATE      CONNECTION
wlan0       wifi      connected  WN-25011421
l4tbr0      bridge    connected (externally)  l4tbr0
docker0     bridge    connected (externally)  docker0
p2p-dev-wlan0 wifi-p2p   disconnected  --
eth0        ethernet  unavailable  --
can0        can        unmanaged    --
usb0        ethernet  unmanaged    --
usb1        ethernet  unmanaged    --
lo          loopback   unmanaged    --
```

3. 在终端输入指令 `sudo nmcli connection down "WN-25011421"` 使WiFi断开连接。

```
> sudo nmcli connection down "WN-25011421"
Connection 'WN-25011421' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/5)
```

4. 进入到 `wifi_control` 文件夹下，并输入指令 `sudo python3 wifi_basic_demo.py`，机械臂将会按照指定坐标进行移动。

```
> sudo python3 wifi_basic_demo.py
[WiFi] connecting to SSID: NexArm
[WiFi] connected to NexArm hotspot
[WiFi] firmware version → 1.0.0
[STEP 1] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=-50.0, time=1500
[STEP 2] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=0.0, time=400
[STEP 3] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=0.0, time=400
[STEP 4] move → x=250.0, y=0.0, z=100.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 5] move → x=200.0, y=0.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 6] move → x=0.0, y=200.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 7] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 8] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=-50.0, time=400
[STEP 9] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=-50.0, time=400
[STEP 10] move → x=0.0, y=250.0, z=100.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 11] move → x=0.0, y=200.0, z=200.0, pitch=0.0, roll=0.0, claw=-50.0, time=1500
[STEP 12] move → x=200.0, y=0.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 12] current coords(from NexArm WiFi) → x=201, y=2, z=189, pitch=-4.3, roll=0, claw=0, servos=(2043, 2169, 1215, 1386, 2047, 2043)
```

6.1.5 程序分析

1. `wifi_basic_demo.py` 中先定义了 WiFi 热点参数、机械臂 TCP 连接地址以及整套搬运动作序列。

`PICK_PLACE_SEQUENCE` 数组中的每一项都包含 `x`、`y`、`z`、`pitch`、`roll`、`claw`、`duration_ms` 七个字段。

```
WIFI_SSID = "NexArm"
WIFI_PASSWORD = "hiwonder"
NEXARM_HOST = "192.168.4.1"
NEXARM_PORT = 8080

PICK_PLACE_SEQUENCE = [
    (250.0, 0.0, 20.0, -90.0, 0.0, -50.0, 1500),
    (250.0, 0.0, 20.0, -90.0, 0.0, 0.0, 400),
    (200.0, 0.0, 200.0, 0.0, 0.0, 0.0, 1500),
]
```

2. 在建立连接之前，程序先调用 `get_active_ssid()`、`get_visible_ssids()` 和 `can_connect_nexarm()` 检查当前网络环境。

```
def connect_nexarm_wifi(ssid=WIFI_SSID, password=WIFI_PASSWORD, wait_seconds=8):
    active_ssid = get_active_ssid()
    if active_ssid == ssid:
        print("[WiFi] already connected to {}".format(ssid))
        return

    if can_connect_nexarm():
        print("[WiFi] NexArm TCP service already reachable, skip reconnect")
        return
```

3. 当树莓派尚未连接到 NexArm 热点时，程序会通过 `nmcli` 扫描并连接指定 WiFi。这里不是直接盲目执行连接，而是先重扫热点列表，再确认 `NexArm` 是否可见，最后才执行 `nmcli dev wifi connect`。

```
for attempt in range(1, 4):
    subprocess.run(
        ["nmcli", "dev", "wifi", "rescan"],
        capture_output=True,
        text=True,
    )
    time.sleep(2)

    visible_ssids = get_visible_ssids()
    if ssid not in visible_ssids:
        print("[WiFi] scan {}: SSID '{}' not found".format(attempt, ssid))
        continue

    result = subprocess.run(
        ["nmcli", "dev", "wifi", "connect", ssid, "password", password],
        capture_output=True,
        text=True,
    )
```

4. `nexarm_wifi_sdk.py` 中的 `NexArmWiFiClient` 负责完成协议打包和网络通信。`build_frame()` 按照 帧头 + ID + 长度 + 命令字 + 载荷 + 校验 的顺序封装数据，`send_frame()` 再通过 `sock.sendall()` 把整帧数据发给机械臂。

```
@classmethod
def build_frame(cls, packet_id, cmd, payload=b''):
    length = len(payload) + 2
    checksum = cls._checksum(packet_id, length, cmd, payload)
    return FRAME_HEADER + bytes([packet_id, length, cmd]) + payload + bytes([checksum])

def send_frame(self, packet_id, cmd, payload=b''):
    sock = self._ensure_connected()
    frame = self.build_frame(packet_id, cmd, payload)
    sock.sendall(frame)
    return frame
```

5. 控制机械臂运动时，`set_pose()` 会把 `pitch` 乘以 10 转成整型，再把 `x`、`y`、`z`、`roll`、`claw` 和运行时间一起按小端格式打包成 `CMD_COORDINATE_SET` 指令。

```
def set_pose(self, x, y, z, pitch, roll, claw, duration_ms):
    payload = (
        self._i16(round(pitch * 10.0))
        + self._i16(round(x))
        + self._i16(round(y))
        + self._i16(round(z))
        + self._i16(round(roll))
        + self._i16(round(claw))
        + self._u16(duration_ms)
    )
    self.request(SYSTEM_ID, NexArmCommand.CMD_COORDINATE_SET, payload, expect_reply=False)
```

6. `main()` 中先完成热点连接, 再创建 `NexArmWiFiClient` 对象, 随后依次执行固件版本读取、蜂鸣器提示和动作序列下发。每走完一组动作后, 程序还会调用 `get_current_coords()` 主动读取当前坐标并打印出来。

```
def main():
    connect_nexarm_wifi()

    with NexArmWiFiClient(host=NEXARM_HOST, port=NEXARM_PORT, timeout=1.0) as arm:
        version = arm.get_firmware_version(timeout=1.0)
        print("[WiFi] firmware version -> {}".format(version))

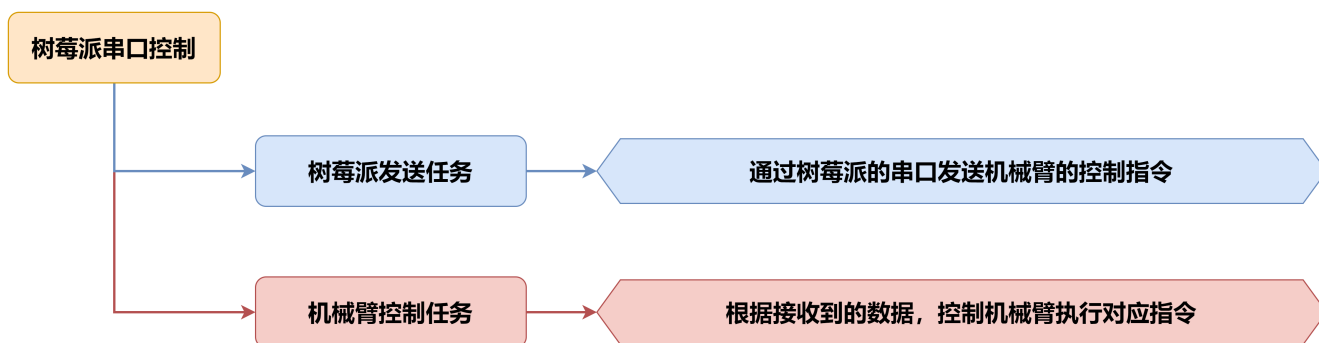
        arm.set_buzzer(100, 50, 1, 2000)
        for index, pose in enumerate(PICK_PLACE_SEQUENCE, start=1):
            x, y, z, pitch, roll, claw, duration_ms = pose
            arm.set_pose(x, y, z, pitch, roll, claw, duration_ms)
            time.sleep(duration_ms / 1000.0)
        print_current_coords(arm, index)
```

6.2 树莓派串口控制机械臂

6.2.1 项目介绍

本节通过树莓派串口与 NexArm 直接通信, 由树莓派侧 Python 程序完成协议封装, 并通过 USB 串口向机械臂发送控制指令。程序运行后, 会先建立与 `/dev/ttyUSB0` 的串口连接, 再依次下发预设搬运动作。

6.2.2 实现流程

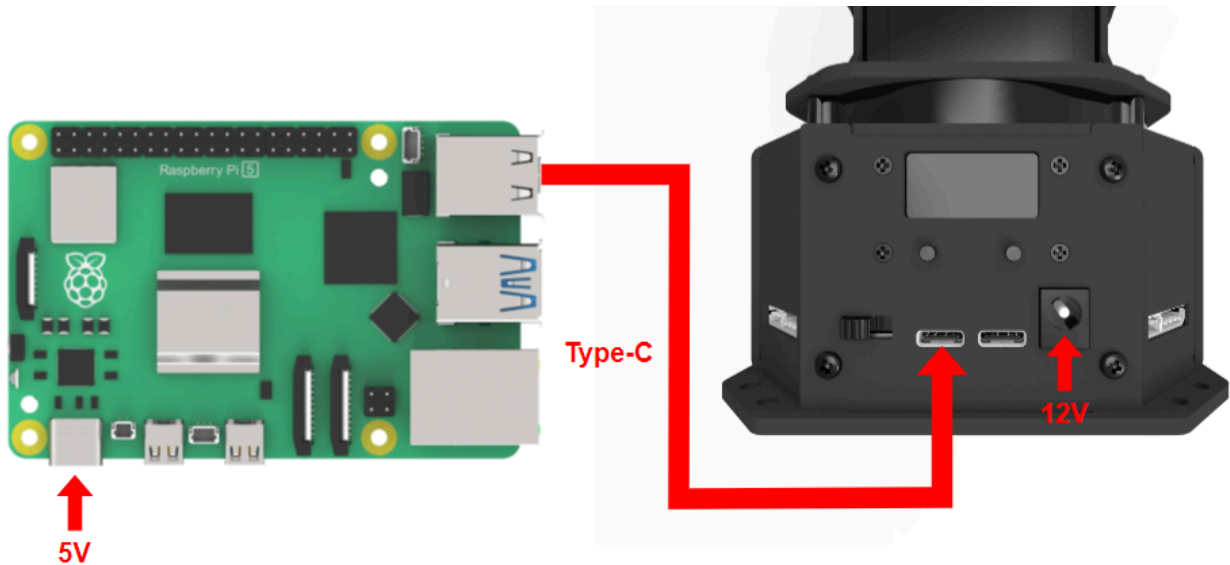


6.2.3 程序下载

参考[程序下载](#)步骤，下载NexArm程序，并将"源码\UART_Control"文件夹，上传至树莓派中。

6.2.4 实现效果

- 1. 使用Type-C数据线将树莓派与NexArm进行连接。



- 2. 进入到 `UART_Control` 文件夹下，并输入指令 `python3 basic_demo.py`，机械臂将会按照指定坐标进行移动。

```
> python3 basic_demo.py
[UART] firmware version → 1.0.0
[STEP 1] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=-50.0, time=1500
[STEP 2] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=0.0, time=400
[STEP 3] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=0.0, time=400
[STEP 4] move → x=250.0, y=0.0, z=100.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 5] move → x=200.0, y=0.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 6] move → x=0.0, y=200.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 7] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 8] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=-50.0, time=400
[STEP 9] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=-50.0, time=400
[STEP 10] move → x=0.0, y=250.0, z=100.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 11] move → x=0.0, y=200.0, z=200.0, pitch=0.0, roll=0.0, claw=-50.0, time=1500
[STEP 12] move → x=200.0, y=0.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 12] current coords(from NexArm) → x=201, y=1, z=190, pitch=-4.3, roll=0, claw=0, servos=(2045, 2172, 1215, 1388, 2046, 2044)
```

6.2.5 程序分析

- 1. `basic_demo.py` 中先用 `PICK_PLACE_SEQUENCE` 定义了整套搬运动作，每个元组都直接对应一条机械臂坐标控制指令。

```
PICK_PLACE_SEQUENCE = [
    (250.0, 0.0, 20.0, -90.0, 0.0, -50.0, 1500),
    (250.0, 0.0, 20.0, -90.0, 0.0, 0.0, 400),
    (200.0, 0.0, 200.0, 0.0, 0.0, 0.0, 1500),
]
```

- 2. `NexArmClient` 在初始化时记录串口号、波特率和超时时间，`open()` 中通过 `serial.Serial(...)` 建立与机械臂的物理串口连接。

```
class NexArmClient:
    def __init__(self, port, baudrate=1000000, timeout=0.1, write_timeout=0.1):
        self.port = port
```

```

self.baudrate = baudrate
self.timeout = timeout
self.write_timeout = write_timeout
self.serial = None

def open(self):
    self.serial = serial.Serial(
        port=self.port,
        baudrate=self.baudrate,
        bytesize=serial.EIGHTBITS,
        parity=serial.PARITY_NONE,
        stopbits=serial.STOPBITS_ONE,
        timeout=self.timeout,
        write_timeout=self.write_timeout,
    )

```

3. 数据发送前，程序先通过 `_checksum()` 计算校验值，再由 `build_frame()` 统一拼装完整协议帧。

```

@staticmethod
def _checksum(packet_id, length, cmd, payload):
    total = packet_id + length + cmd + sum(payload)
    return (~total) & 0xFF

@classmethod
def build_frame(cls, packet_id, cmd, payload=b''):
    length = len(payload) + 2
    checksum = cls._checksum(packet_id, length, cmd, payload)
    return FRAME_HEADER + bytes([packet_id, length, cmd]) + payload + bytes([checksum])

```

4. 串口接收部分由 `read_packet()` 负责，它按状态机方式逐字节寻找帧头、读取长度、解析命令和载荷，最后再校验校验和。

```

while time.monotonic() < deadline:
    chunk = ser.read(1)
    if not chunk:
        continue
    byte = chunk[0]

    if state == 0:
        if byte == 0xFF:
            state = 1
        continue

```

5. 机械臂坐标控制通过 `set_pose()` 完成。函数内部把 `pitch`、`x`、`y`、`z`、`roll`、`claw` 和 `duration_ms` 按协议要求依次打包，并以 `CMD_COORDINATE_SET` 命令发给下位机。

```
def set_pose(self, x, y, z, pitch, roll, claw, duration_ms):
    payload = (
        self._i16(round(pitch * 10.0))
        + self._i16(round(x))
        + self._i16(round(y))
        + self._i16(round(z))
        + self._i16(round(roll))
        + self._i16(round(claw))
        + self._u16(duration_ms)
    )
    self.request(SYSTEM_ID, NexArmCommand.CMD_COORDINATE_SET, payload, expect_reply=False)
```

6. 在动作演示部分，程序先读取固件版本，再触发一次蜂鸣器，接着顺序执行动作数组中的所有坐标。动作全部完成后，`print_current_coords()` 会调用 `get_current_coords()` 读取机械臂当前实际坐标，并把末端姿态和六个舵机位置一起打印出来。

```
with NexArmClient("/dev/ttyUSB0") as arm:
    version = arm.get_firmware_version(timeout=1.0)
    print("[UART] firmware version -> {0}".format(version))

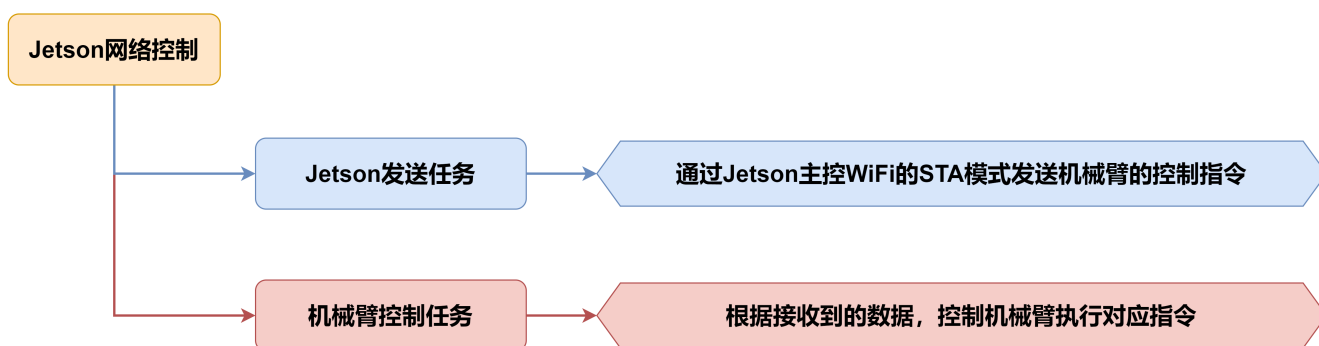
    arm.set_buzzer(on_ms=100, off_ms=50, times=1, frequency=2000)
    for index, pose in enumerate(PICK_PLACE_SEQUENCE, start=1):
        x, y, z, pitch, roll, claw, duration_ms = pose
        arm.set_pose(x, y, z, pitch, roll, claw, duration_ms)
        time.sleep(duration_ms / 1000.0)
    print_current_coords(arm, index)
```

6.3 Jetson网络控制机械臂

6.3.1 项目介绍

本节通过 Jetson 平台连接 NexArm 热点，并使用 Python 程序通过 TCP 网络控制机械臂。Jetson 端程序会先检查当前无线连接状态，确认已经接入 NexArm 热点后，再与机械臂开放的网络端口建立连接，随后按照预设搬运路径依次下发坐标指令，实现远程网络控制。

6.3.2 实现流程



6.3.3 程序下载

- NexArm程序下载

机械臂在进行外设主控控制功能前，需要先下载对应的控制程序，程序下载可参考以下步骤：

1. 使用Type-C数据线将NexArm与电脑进行连接。



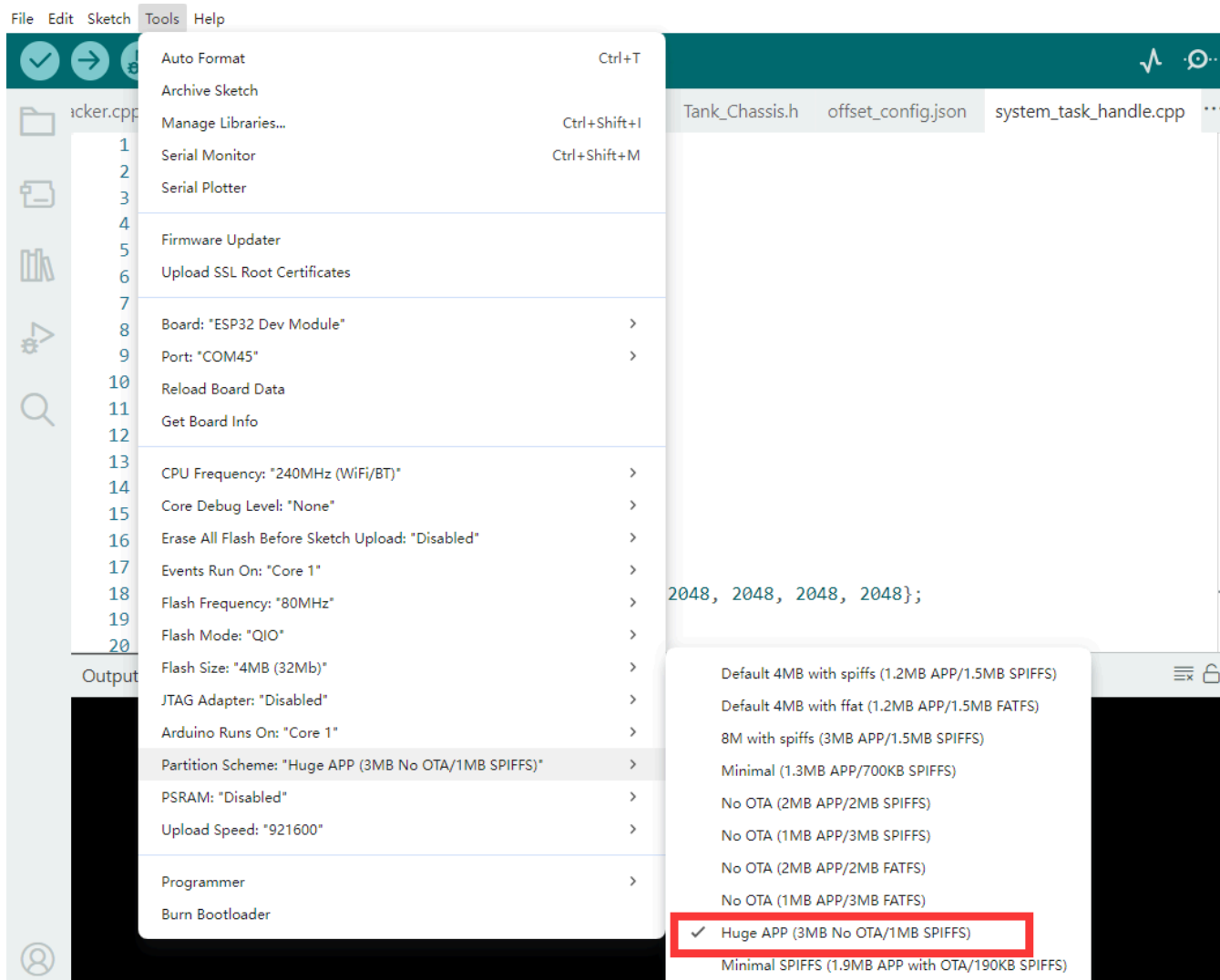
2. 打开本文档同路径下"源码"中对应的ino程序文件。



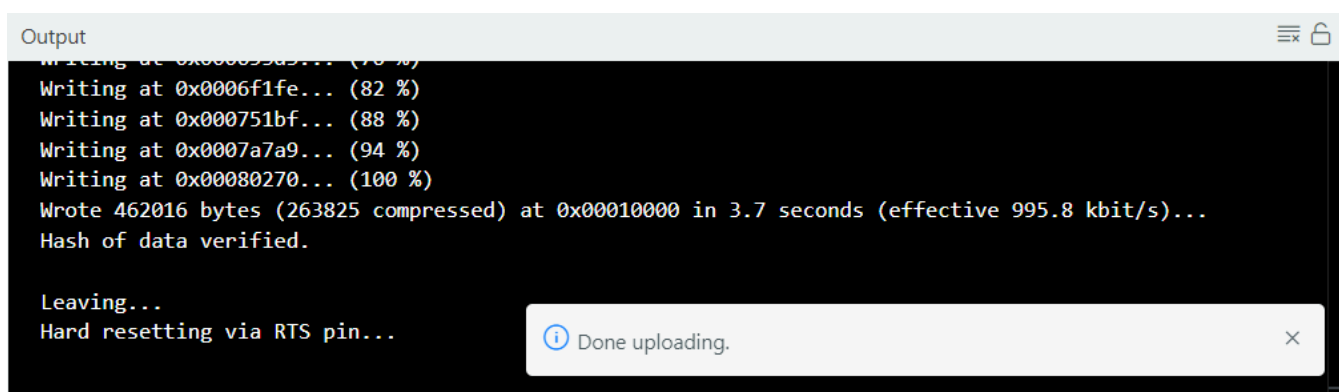
3. 打开后，选择开发板型号，具体型号如下图所示：



4. 点击菜单栏的"工具", 按照下图选择相应的ESP32的开发板配置：



5. 先点击"编译", 再点击"上传", 当软件下方输出框出现如下界面, 说明程序下载成功:



• Jetson程序下载

使用远程工具将"源码\WiFi_Control"文件夹, 上传至Jetson主控中。

6.3.4 实现效果

1. 首先使用Type-C线连接主板与Jetson主控, 并在终端输入 `sudo ifconfig` 指令查找 eth0 的IP地址, 并使用远程工具通过有线IP地址进行连接。

```
> sudo ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.11.138 netmask 255.255.254.0 broadcast 192.168.11.255
    inet6 fe80::84ee:1627:16f2:b04c prefixlen 64 scopeid 0x20<link>
    ether 2c:cf:67:68:a9:37 txqueuelen 1000 (Ethernet)
    RX packets 10812 bytes 1508077 (1.4 MiB)
    RX errors 0 dropped 153 overruns 0 frame 0
    TX packets 741 bytes 94625 (92.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 106
```

2. 接着输入 `nmcli device status` 指令查询 `wlan0` 的连接状态，其中 "WN-25011421" 为网络连接名称。

```
> nmcli device status
DEVICE      TYPE      STATE      CONNECTION
wlan0       wifi      connected  WN-25011421
l4tbr0      bridge   connected (externally)  l4tbr0
docker0     bridge   connected (externally)  docker0
p2p-dev-wlan0 wifi-p2p   disconnected  --
eth0        ethernet unavailable  --
can0        can       unmanaged   --
usb0        ethernet unmanaged   --
usb1        ethernet unmanaged   --
lo          loopback  unmanaged   --
```

3. 在终端输入指令 `sudo nmcli connection down "WN-25011421"` 使WiFi断开连接。

```
> sudo nmcli connection down "WN-25011421"
Connection 'WN-25011421' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/5)
```

4. 进入到 `wifi_control` 文件夹下，并输入指令 `sudo python3 wifi_basic_demo.py`，机械臂将会按照指定坐标进行移动。

```
> sudo python3 wifi_basic_demo.py
[WiFi] connecting to SSID: NexArm
[WiFi] connected to NexArm hotspot
[WiFi] firmware version → 1.0.0
[STEP 1] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=-50.0, time=1500
[STEP 2] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=0.0, time=400
[STEP 3] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=0.0, time=400
[STEP 4] move → x=250.0, y=0.0, z=100.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 5] move → x=200.0, y=0.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 6] move → x=0.0, y=200.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 7] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 8] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=-50.0, time=400
[STEP 9] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=-50.0, time=400
[STEP 10] move → x=0.0, y=250.0, z=100.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 11] move → x=0.0, y=200.0, z=200.0, pitch=0.0, roll=0.0, claw=-50.0, time=1500
[STEP 12] move → x=200.0, y=0.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 12] current coords(from NexArm WiFi) → x=201, y=2, z=189, pitch=-4.3, roll=0, claw=0, servos=(2043, 2169, 1215, 1386, 2047, 2043)
```

6.3.5 程序分析

1. Jetson 网络控制部分与树莓派网络控制使用同一套 `wifi_control` 程序结构，动作序列同样由

`PICK_PLACE_SEQUENCE` 给出。

```
PICK_PLACE_SEQUENCE = [
    (250.0, 0.0, 20.0, -90.0, 0.0, -50.0, 1500),
    (250.0, 0.0, 20.0, -90.0, 0.0, 0.0, 400),
    (200.0, 0.0, 200.0, 0.0, 0.0, 0.0, 1500),
]
```

2. 网络准备阶段通过 `connect_nexarm_wifi()` 统一处理。函数先检测当前是否已经连接到目标热点，再检查 NexArm 的 TCP 服务是否已经可达；只有两者都不满足时，才会调用 `nmcli` 重新扫描和连接热点。

```

active_ssid = get_active_ssid()
if active_ssid == ssid:
    print("[WiFi] already connected to {}".format(ssid))
    return

if can_connect_nexarm():
    print("[WiFi] NexArm TCP service already reachable, skip reconnect")
    return

```

3. `NexArmWiFiClient` 把 TCP 套接字通信和协议解析集中在同一个类里。`connect()` 负责创建 socket 并连接机械臂热点地址，`send_frame()` 负责把封装好的协议帧发送出去。

```

def connect(self):
    if self.sock:
        return
    self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    self.sock.settimeout(self.timeout)
    self.sock.connect((self.host, self.port))

def send_frame(self, packet_id, cmd, payload=b''):
    sock = self._ensure_connected()
    frame = self.build_frame(packet_id, cmd, payload)
    sock.sendall(frame)
    return frame

```

4. WiFi 接收数据时，程序不是每次都假定 `recv()` 刚好读到一整帧，而是先把收到的数据累计到 `recv_buffer`，再由 `_try_parse_packet_from_buffer()` 从缓冲区中按帧头、长度和校验和进行拆包。

```

def read_packet(self, timeout=None):
    while time.monotonic() < deadline:
        packet = self._try_parse_packet_from_buffer()
        if packet is not None:
            return packet

        data = sock.recv(256)
        self.recv_buffer.extend(data)

```

5. 机械臂当前坐标读取由 `get_current_coords()` 完成。函数在收到回包后，会先解析 `x、y、z`，再根据载荷长度继续解析 `pitch、roll、claw` 以及六个舵机位置。

```

def get_current_coords(self, timeout=1.0):
    packet = self.request(
        SYSTEM_ID,
        NexArmCommand.CMD_GET_CUR_COORDS,
        expect_reply=True,
        expected_cmd=NexArmCommand.CMD_GET_CUR_COORDS,
        expected_ids=(0x5A, SYSTEM_ID),
        timeout=timeout,
    )

```

6. `main()` 中先完成 WiFi 连接，再创建 `NexArmWiFiClient` 对象，随后执行固件版本读取、蜂鸣器提示、动作数组遍历和当前坐标查询。

```
def main():
    connect_nexarm_wifi()

    with NexArmWiFiClient(host=NEXARM_HOST, port=NEXARM_PORT, timeout=1.0) as arm:
        version = arm.get_firmware_version(timeout=1.0)
        print("[WiFi] firmware version -> {}".format(version))

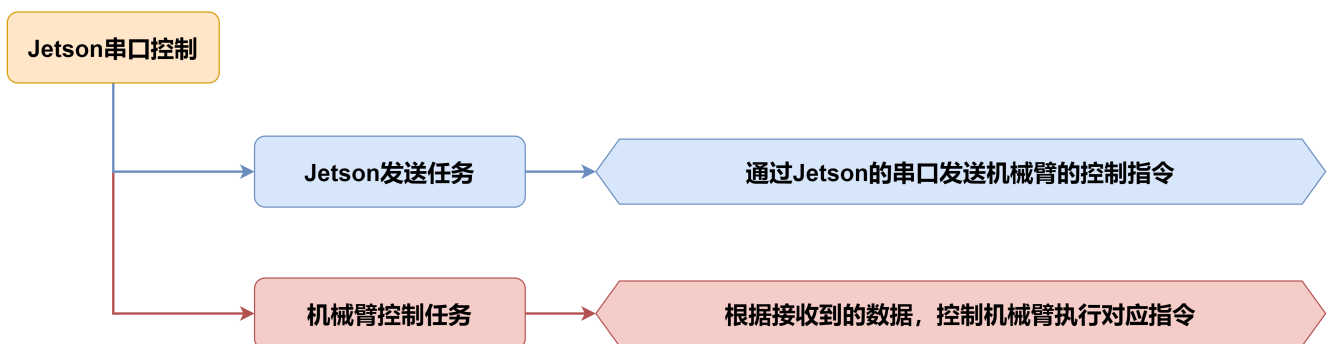
    arm.set_buzzer(100, 50, 1, 2000)
    for index, pose in enumerate(PICK_PLACE_SEQUENCE, start=1):
        x, y, z, pitch, roll, claw, duration_ms = pose
        arm.set_pose(x, y, z, pitch, roll, claw, duration_ms)
        time.sleep(duration_ms / 1000.0)
```

6.4 Jetson串口控制机械臂

6.4.1 项目介绍

本节通过 Jetson 与 NexArm 的 USB 串口建立直接通信，由 Jetson 端 Python 程序完成协议数据的封装、发送和回读。程序运行后，会先打开对应串口，再依次向机械臂下发预设坐标控制命令，实现机械臂的定点抓取与搬运。

6.4.2 实现流程

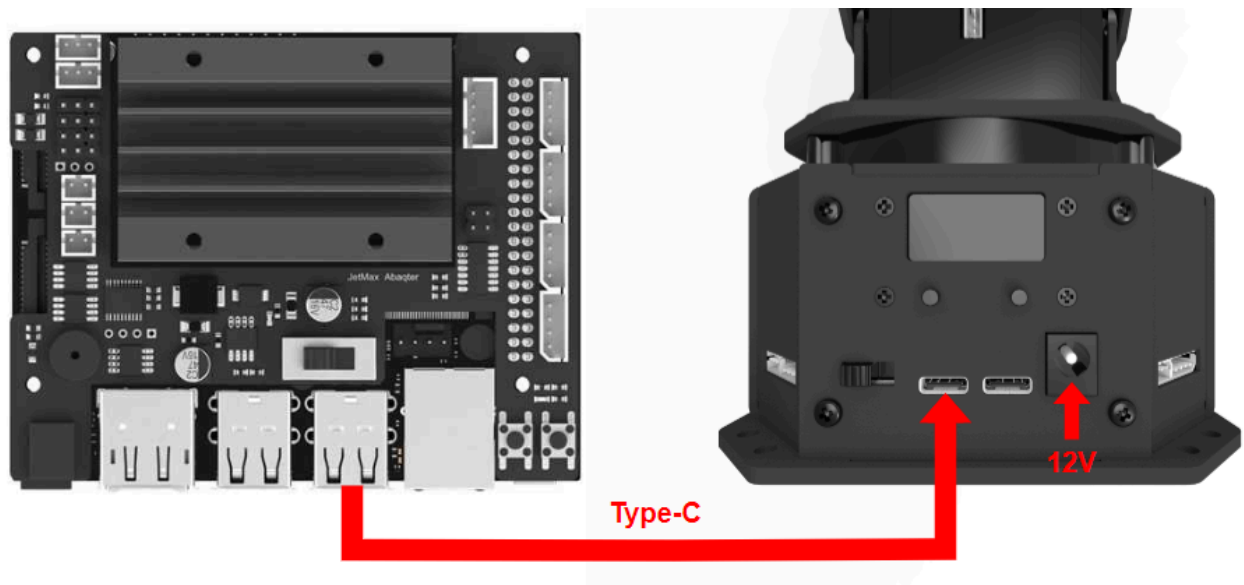


6.4.3 程序下载

参考[程序下载](#)步骤，下载NexArm程序，并将"源码\UART_Control"文件夹，上传至Jetson主控中。

6.4.4 实现效果

1. 使用Type-C数据线将Jetson主控与NexArm进行连接。



2. 进入到 `UART_Control` 文件夹下，并输入指令 `python3 basic_demo.py`，机械臂将会按照指定坐标进行移动。

```
> python3 basic_demo.py
[UART] firmware version → 1.0.0
[STEP 1] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=-50.0, time=1500
[STEP 2] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=0.0, time=400
[STEP 3] move → x=250.0, y=0.0, z=20.0, pitch=-90.0, roll=0.0, claw=0.0, time=400
[STEP 4] move → x=250.0, y=0.0, z=100.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 5] move → x=200.0, y=0.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 6] move → x=0.0, y=200.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 7] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 8] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=-50.0, time=400
[STEP 9] move → x=0.0, y=250.0, z=40.0, pitch=-90.0, roll=0.0, claw=-50.0, time=400
[STEP 10] move → x=0.0, y=250.0, z=100.0, pitch=-90.0, roll=0.0, claw=0.0, time=1500
[STEP 11] move → x=0.0, y=200.0, z=200.0, pitch=0.0, roll=0.0, claw=-50.0, time=1500
[STEP 12] move → x=200.0, y=0.0, z=200.0, pitch=0.0, roll=0.0, claw=0.0, time=1500
[STEP 12] current coords(from NexArm) → x=201, y=1, z=190, pitch=-4.3, roll=0, claw=0, servos=(2045, 2172, 1215, 1388, 2046, 2044)
```

6.4.5 程序分析

1. Jetson 串口控制部分与树莓派串口控制共用 `UART_Control` 程序。动作参数同样通过 `PICK_PLACE_SEQUENCE` 统一定义，数组中每一项都包含完整的目标坐标、末端姿态、夹爪角度和运行时间。

```
PICK_PLACE_SEQUENCE = [
    (250.0, 0.0, 20.0, -90.0, 0.0, -50.0, 1500),
    (250.0, 0.0, 20.0, -90.0, 0.0, 0.0, 400),
    (200.0, 0.0, 200.0, 0.0, 0.0, 0.0, 1500),
]
```

2. `NexArmClient` 先通过 `serial.serial(...)` 打开主控与机械臂之间的串口，再在整个控制过程中复用这一条通信链路。

```
def open(self):
    if self.serial and self.serial.is_open:
        return
    self.serial = serial.Serial(
        port=self.port,
        baudrate=self.baudrate,
        bytesize=serial.EIGHTBITS,
        parity=serial.PARITY_NONE,
        stopbits=serial.STOPBITS_ONE,
        timeout=self.timeout,
        write_timeout=self.write_timeout,
    )
```

3. 协议发送部分由 `build_frame()` 和 `send_frame()` 配合完成。 `build_frame()` 根据命令字和载荷生成标准通信帧， `send_frame()` 再把整帧写入串口并立即 `flush()`。

```
def send_frame(self, packet_id, cmd, payload=b''):
    ser = self._ensure_open()
    frame = self.build_frame(packet_id, cmd, payload)
    ser.write(frame)
    ser.flush()
    return frame
```

4. 回复数据的读取由 `read_packet()` 负责。函数使用状态机逐字节读取数据，并在收到完整一帧后校验校验和；如果校验失败，就立即抛出异常。

```
if state == 6:
    checksum = byte
    expected = self._checksum(packet_id, length, cmd, bytes(payload))
    if checksum != expected:
        raise ValueError(
            "Checksum mismatch: got 0x{0:02X}, expected 0x{1:02X}".format(
                checksum, expected
            )
        )
```

5. SDK 对上层控制接口做了进一步封装，例如 `get_firmware_version()` 负责读取三字节固件版本， `set_buzzer()` 负责控制蜂鸣器， `get_current_coords()` 负责读取当前位置， `set_pose()` 则负责发送坐标控制命令。

```
def get_firmware_version(self, timeout=0.5):
    packet = self.request(
        SYSTEM_ID,
        NexArmCommand.CMD_FIRMWARE_VERSION_CHECK,
        expect_reply=True,
        expected_cmd=NexArmCommand.CMD_FIRMWARE_VERSION_CHECK,
        expected_ids=(SYSTEM_ID, 0x5A),
        timeout=timeout,
    )
```

6. 主程序中，Jetson 端先读取固件版本，再触发一次蜂鸣器提示，随后遍历动作数组依次控制机械臂运动。所有动作完成后，再调用 `print_current_coords()` 读取一次机械臂的实际坐标并打印出来。

```
def main():
    with NexArmClient("/dev/ttyUSB0") as arm:
        version = arm.get_firmware_version(timeout=1.0)
        print("[UART] firmware version -> {}".format(version))

        arm.set_buzzer(on_ms=100, off_ms=50, times=1, frequency=2000)
        for index, pose in enumerate(PICK_PLACE_SEQUENCE, start=1):
            x, y, z, pitch, roll, claw, duration_ms = pose
            arm.set_pose(x, y, z, pitch, roll, claw, duration_ms)
            time.sleep(duration_ms / 1000.0)
        print_current_coords(arm, index)
```